

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

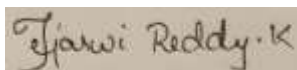
Constraint-Based Problem Solving

Code of Conduct:

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

SOLUTION –

Algorithm Used

Backtracking Search

Backtracking is a depth-first search algorithm that assigns values one by one and checks whether the current assignment satisfies all constraints. If any constraint is violated, the algorithm backtracks and tries another assignment.

Variables

- D1
- D2
- D3

Domain (Possible Shifts)

- Morning (M)
- Afternoon (A)
- Night (N)

Initial State

No doctor is assigned any shift.

D1 = -

D2 = -

D3 = -

Constraint Checking

- D1 ≠ Night
- D3 ≠ Morning
- D2 must be scheduled before D3
- No two doctors can have the same shift
- Every doctor gets exactly one shift

Backtracking Search Steps

Step 1

Assign **D1 = Morning**

Constraint Check:

- D1 is not Night ✓

Current Assignment:

D1 = Morning
D2 = -
D3 = -

Step 2

Assign **D2 = Afternoon**

Constraint Check:

- Different from D1 ✓
- D2 is before D3 (possible) ✓

Current Assignment:

D1 = Morning
D2 = Afternoon
D3 = -

Step 3

Assign **D3 = Night**

Constraint Check:

- D3 is not Morning ✓

- D2 (Afternoon) comes before D3 (Night) ✓
- All shifts are unique ✓
- Every doctor has exactly one shift ✓

All constraints are satisfied.

Search Tree

```

Start
|
D1 = Morning
|
D2 = Afternoon
|
D3 = Night
|
Goal State

```

Constraint Verification

Constraint	Status
D1 cannot work Night	✓ Satisfied
D2 works before D3	✓ Satisfied
One doctor per shift	✓ Satisfied
D3 cannot work Morning	✓ Satisfied
Every doctor assigned exactly one shift	✓ Satisfied

Final Valid Schedule

Doctor Shift

```

D1    Morning
D2    Afternoon
D3    Night

```

Pseudocode

START

Assign shifts to doctors one by one

Assign D1

```

IF D1 = Night
    Backtrack

```

Assign D2

```

IF shift already assigned
    Backtrack

```

Assign D3

```

IF D3 = Morning
    Backtrack

```

```
IF D2 is after D3
    Backtrack

IF all constraints are satisfied
    Print Schedule
ELSE
    Try another assignment

STOP
```

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

SOLUTION –

Grid Representation

```
S . . X .
. X . X .
. X . . .
. . X X .
. . . . G
```

Where:

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- **.** = Free Cell

Algorithm Used

Breadth-First Search (BFS)

BFS explores nodes level by level and guarantees the shortest path when all edge costs are equal.

Step-by-Step Node Expansion

Step 1

Start at **S (0,0)**

Queue:

[(0,0)]

Expand:

(0,0)

New Queue:

[(0,1), (1,0)]

Step 2

Expand **(0,1)**

New Queue:

[(1,0), (0,2)]

Step 3

Expand **(1,0)**

New Queue:

[(0,2), (2,0)]

Step 4

Expand **(0,2)**

New Queue:

[(2,0), (1,2)]

Step 5

Expand **(2,0)**

New Queue:

[(1,2), (3,0)]

Step 6

Continue expanding nodes level by level until the Goal node is reached.

Final expansion reaches:

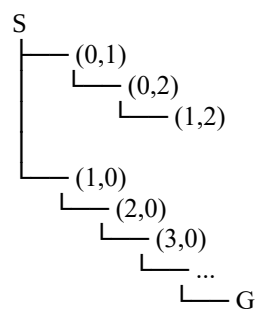
Goal (4,4)

Queue Updates

Step Expanded Node Queue After Expansion

1	(0,0)	[(0,1), (1,0)]
2	(0,1)	[(1,0), (0,2)]
3	(1,0)	[(0,2), (2,0)]
4	(0,2)	[(2,0), (1,2)]
5	(2,0)	[(1,2), (3,0)]
...	Continue BFS	Goal Reached

Search Tree



Optimal Path

S
→ (1,0)
→ (2,0)
→ (3,0)
→ (4,0)
→ (4,1)
→ (4,2)
→ (4,3)
→ G (4,4)

Cost Calculation

Each move has a cost of **1**.

Number of Moves = 8

Total Cost = $8 \times 1 = 8$

Manhattan Distance

The Manhattan Distance heuristic is

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Although provided in the problem statement, **BFS does not use the Manhattan heuristic**. It explores nodes in level order using a FIFO queue.

Pseudocode

START

Create an empty Queue

Insert Start Node into Queue

Mark Start as Visited

WHILE Queue is not Empty

 Remove Front Node

 IF Current Node is Goal

 Print Path

 STOP

 FOR each valid neighbour

 IF neighbour is not visited AND not an obstacle

 Mark Visited

 Insert into Queue

 ENDIF

 ENDFOR

ENDWHILE

STOP

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

SOLUTION –

(a) Analysis of Constraints

Constraint	Effect on Decision Making
Move only Up, Down, Left, Right	The robot explores only four neighbouring cells.
Obstacles cannot be crossed	Invalid paths are discarded during search.
Limited sensing capability	The robot knows only nearby cells and must explore gradually.
Movement cost = 1	Every movement increases the path cost by one.
Risky zone cost = +2	The robot avoids risky areas unless necessary.
Minimize total cost	The robot selects the least-cost path rather than the shortest distance alone.
Dynamic knowledge update	Newly discovered information is used to re-plan the route whenever required.

The robot continuously:

1. Observes neighbouring rooms.
2. Detects obstacles and risky zones.
3. Updates its internal map.
4. Chooses the least-cost safe path.
5. Re-plans whenever new obstacles are discovered

(b) Solution Approach

Search Strategy Used -**Online Search Agent with Uniform Cost Search (UCS)**

Reason

Since the robot does not know the complete environment initially, it behaves as an **Online Search Agent**. Whenever new cells become visible, it updates the map and applies **Uniform Cost Search** to determine the minimum-cost path.

Algorithm Steps

1. Start at **S**.
2. Sense adjacent cells.
3. Update the internal map.
4. Ignore blocked cells.
5. Compute the cumulative cost:
 - Normal move = 1
 - Risky cell = 3 (1 + 2)
6. Expand the node having the minimum total cost.
7. Continue until the survivor (**G**) is reached.
8. If new obstacles are detected, update the map and search again.

Pseudocode

START

Place Robot at Start (S)

WHILE Goal not reached

 Sense Adjacent Cells

 Update Environment Map

 Remove Blocked Paths

 Calculate Cost of Each Possible Move

 Select Path with Minimum Cost

 Move to Selected Cell

END WHILE

Rescue Survivor

STOP

(c) Application of AI Concepts

1. Intelligent Agent

The rescue robot acts as an **Intelligent Agent** because it:

- Perceives the environment using sensors.
- Processes the collected information.
- Makes autonomous decisions.
- Performs actions to achieve its goal.

PEAS Representation

Component	Description
Performance Measure	Rescue survivor safely with minimum cost and time
Environment	Disaster-affected building
Actuators	Wheels, robotic arm, communication system
Sensors	Camera, LIDAR, ultrasonic sensor, infrared sensor

2. Rationality

The robot is **rational** because it always selects the action that maximizes the chance of successfully rescuing the survivor while minimizing:

- Travel cost
- Risk
- Time

It avoids dangerous paths whenever a safer alternative exists.

3. Environment Type

Property	Description
Observability	Partially Observable
Determinism	Stochastic (uncertain environment)
Episodic / Sequential	Sequential
Static / Dynamic	Dynamic
Discrete / Continuous	Discrete Grid
Single / Multi-Agent	Single Agent
Known / Unknown	Unknown Environment

(d) Justification of the Chosen Approach

The combination of an **Online Search Agent** and **Uniform Cost Search (UCS)** is the most suitable because:

- The environment is initially unknown.
- The robot discovers new information while moving.
- UCS guarantees the least-cost path considering both movement and risk costs.
- The robot can dynamically update its knowledge and re-plan when obstacles are detected.
- It provides safe and efficient navigation, which is essential during rescue operations.
- Adapts to changing environments.
- Guarantees optimal cost using Uniform Cost Search.
- Efficient for autonomous rescue operations.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided